



How Data Systems Shape AI Agent Efficiency and Accuracy

Shannon Kang, Hengrui Zhang, Haocheng Xia, Yongjoo Park

University of Illinois Urbana-Champaign

sk227@illinois.edu, hengrui7@illinois.edu, hxia7@illinois.edu, yongjoo@illinois.edu

🔗 Source Code: <https://github.com/illinoisdata/mini-data-agent>

Abstract

An AI agent’s ability to explore and analyze data depends critically on the underlying data system. Yet, no existing work systematically studies how data system architecture affects agent performance. We empirically evaluate AI agents on three common data systems: (i) local file systems, (ii) cloud object storage (S3), and (iii) distributed systems with co-located compute (Ray). Using mini-SWE-agent with GPT-5.4 and GPT-5.4-Mini on three benchmarks (LakeQA-mini, KramaBench, and DAB), we find that accuracy remains largely stable across architectures (differences within run-to-run variance), but runtime and I/O costs differ substantially. Moving data to S3 increases command execution time, with data loading accounting for 55–87% of command time on file-heavy benchmarks. LLM latency accounts for only 43–74% of total runtime, less than expected. Agent trajectories vary more by model and task type than by data system: GPT-5.4-Mini takes more steps but writes shorter commands than GPT-5.4. Co-located compute (Ray) is rarely used effectively, with Ray jobs accounting for only 1.8% of commands and exhibiting the highest failure rate (25.0%). Our results demonstrate that while agents adapt their access strategies without losing accuracy, data system architecture significantly impacts efficiency, and current agents fail to exploit co-located compute opportunities.

Keywords: AI agents, data systems, benchmarking

1. Introduction

An AI agent’s ability to explore and analyze data is shaped by the data system underneath it. Before an agent can produce the final answer, it must find the relevant data, decide how to read it, and decide where to run its analysis scripts. These choices depend on whether the data sits on a local file system, behind an object-store API such as Amazon S3, inside SQL databases (e.g., PostgreSQL), or in a remote system (e.g., Ray [18]) that can execute computation next to the data. Real-world deployments span all of these settings, and each exposes different commands and costs, thus causing different failure modes for the agent.

Despite this, no existing work systematically studies how the underlying data system affects an AI agent’s performance. Recent benchmarks evaluate agents on search, multi-step analysis, and multi-database querying [22, 12, 17], but they largely assume data stored in local files and judge agents on answer accuracy. Work that goes beyond accuracy to report monetary cost and latency [10, 11, 13] treats data systems as a fixed background rather than a variable. Computation pushdown and near-data processing show that where computation runs changes how much data moves [30, 28], but they have not been examined for agents that choose their own access strategy dynamically.

In this work, we empirically evaluate an AI agent’s performance (accuracy, efficiency, and I/O costs) on three common data systems: (i) a local file system, (ii) cloud storage behind an S3 interface, and (iii) a remote data system with co-located compute, where the agent may either fetch data or ship code to run next to it. We hold the agent harness (mini-SWE-agent [27]) and tasks fixed. Prompts are largely identical except for a few lines of code explaining data access methods specific to where the data is stored. We evaluate GPT-5.4 and GPT-5.4-Mini on LakeQA-mini [22], KramaBench [12], and DAB [17], running each configuration three times.

Our study shows the following findings:

- **Accuracy is largely independent of the data system (Section 4.1).** Across the three data systems, accuracy differences mostly fall within run-to-run variance. For example, GPT-5.4 scores 73.8%, 73.7%, and 70.1% on LakeQA-mini for local, S3, and S3 & Ray. The agent adapts its access strategy to the data system without losing accuracy.
- **Model differences stem from consistency, not capability (Section 4.1).** At the task level, GPT-5.4 solves 62/135, 54/104, and 10/54 tasks in all nine runs on LakeQA-mini, KramaBench, and DAB. GPT-5.4-Mini fails largely the same tasks but is less consistent across runs, succeeding on a task in some runs but failing it in others. Task outcomes show little dependence on the data system.
- **End-to-end time differs substantially, and command execution is a large share of it (Section 4.2).** Moving data to S3 adds loading time, and total command time grows across the three data systems in five of six model–benchmark pairs. Under S3-based setups, loading accounts for 55–87% of command time on LakeQA-mini. LLM latency accounts for only 43–74% of total runtime, which is less than we expected: command execution makes up the remaining 26–57%.
- **Agent trajectories depend on the model and the task, not just the data system (Section 4.2).** GPT-5.4-Mini takes more steps but writes shorter commands (e.g., 14.8 steps $\times$ 520 characters versus 11.1 $\times$ 824 on LakeQA-mini), and writes less code in total. Search-heavy LakeQA-mini yields many short steps, while SQL-driven DAB yields few long ones.
- **The agent’s read method depends on file size (Section 4.3).** Small files are mostly downloaded or streamed, while the share of in-process reads (reading directly from S3 URIs within libraries like DuckDB or pandas) grows with file size. With co-located compute, in-process reads become the dominant way to read large files, and the percentage of downloads to local files decreases across all file sizes.
- **Co-located compute is rarely used effectively (Section 4.3).** Ray jobs have the highest failure rate of any access method (25.0%), and they account for only 1.8% of commands on LakeQA-mini with GPT-5.4. Some jobs never use Ray at all (e.g., a plain DuckDB query). Agents with Ray often receive *more* data than with S3 alone (e.g., 146 GB versus 69 GB for GPT-5.4 on LakeQA-mini), so co-located compute did not reduce data movement.

2. Related Work

LLM Agents and Scaffolds LLM agents solve tasks by alternating between reasoning and acting in an environment, a pattern introduced by ReAct [29] and extended by Reflexion [20] and CodeAct [23]. In practice, this pattern is realized by agent scaffolds, harness code that mediates the model’s access to an execution environment. Examples include SWE-agent [26], OpenHands [24], CrewAI [3], smolagents [19], and mini-SWE-agent [27], which impose varying constraints on how the model plans and executes tool calls. Applied to data tasks, such agents have been surveyed and classified by degree of independence [16]. However, how the data storage architecture itself limits or alters these reasoning loops remains unexamined. Our work fills this gap by evaluating a matrix of models, agents, and EQAbenchmarks across three data environments: local storage, S3 object storage, and distributed execution via Ray Data. Specifically, we profile how these data storage architectures alter agent execution pathways and quantify the resulting I/O costs at scale.

Benchmarks for Data Agents Benchmarks for data agents largely emphasize downstream execution. Works such as InfiAgent-DABench [7] and DSbench [9] evaluate statistical analysis and modeling, while DA-Code [8], ScienceAgentBench [2], DABstep [6], and DAComp [14] test code generation and multi-step workflows. However, these benchmarks all provide the required files upfront. By handing the agent its data directly, they leave its search capabilities unexamined.

To better reflect real-world data analysis, recent works increasingly evaluate search and task execution in tandem. Benchmarks such as CoDA-Bench [35], DataClawBench [33], and LakeQuest [21] test this by embedding target files among distractors. We evaluate three benchmarks from this class: KramaBench [12], LakeQA [22], and DAB [17]. Together, they cover diverse storage setups, including large data lakes, multi-source files, and relational database joins.

Computation Pushdown Reducing data movement by relocating computation toward data is a long-standing concern in data systems. Distributed execution frameworks exploit data locality [4, 31, 32], while near-data

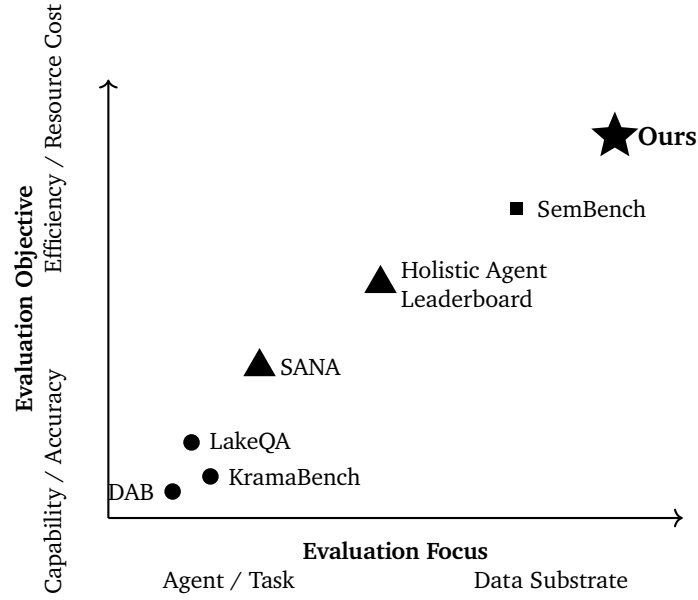


Figure 1: Conceptual landscape of data-agent evaluation. Prior work primarily studies agent capabilities or general execution costs, while our evaluation focuses on the efficiency implications of the underlying data substrate. (briefly, accuracy oriented- $\rightarrow$ efficiency oriented; agent centric - $\rightarrow$ data substrate centric)

processing moves computation closer to storage [1, 5]. Disaggregated cloud storage has renewed interest in these ideas. PushdownDB [30] executes filters, projections, and aggregations within S3 to reduce data transfer and cost. Recent work further studies pushdown in cloud OLAP systems [28], near-data processing in cloud databases such as Taurus [15], and predicate pushdown in general-purpose pandas and Spark pipelines [34].

These studies show how computation placement impacts data movement in standard workloads. We extend this analysis to agentic QA, measuring how offloading computation to a remote cluster affects I/O costs during dynamic data analysis.

Cost and Diagnosis in Agent Evaluation Agent evaluation has moved beyond accuracy alone. Kapoor et al. [10] argue that reporting accuracy in isolation rewards needlessly expensive systems, and the Holistic Agent Leaderboard [11] provides standardized infrastructure for tracking cost alongside it. This multi-dimensional framework has since extended to data settings. SemBench [13] evaluates semantic query engines on quality, monetary cost, and latency jointly, mapping the distinct performance-versus-cost trade-offs across different architectures. SANA [25] takes a diagnostic approach, idealizing an agent’s search, planning, and analysis components in turn to attribute end-to-end performance to each. In this paper, we focus on evaluating the agent based on the data I/O costs across different data storage architectures.

3. Methodology (Sky & Haocheng)

Here, we study the data movement created by an agent as it solves a variety of EQA tasks. Specifically, we measure the bytes that cross from storage to compute across different task styles, storage systems, and execution backends. As AI data agents are increasingly deployed to automate real-world analytics, overall efficiency is influenced not only by model reasoning but also by data access patterns. As a result, understanding the I/O costs associated with these access patterns is critical. Two agents can reach the exact same answer while moving vastly different amounts of data, depending on what they inspect and whether processing happens near the data.

3.1. Benchmarks (1 pg, excluding figure)

TODO: turn the quadrant into a table; columns are features, rows are methods

Benchmark	Tasks	Files	Size	File types	Example task (answer)
LakeQA [22]-mini	135	906	14.3 GB	txt, json, xlsx, pdf, csv, html, docx	For each year 2016–2023, compute annual U.S. drug-overdose deaths $\times (1 + \text{YoY growth})$; report the highest year. (2021)
Kramabench [?]	104	1,742	667 MB	csv, xlsx, sp3, hdr, tle, gpkg, npz, txt	Average age of patients with serous tumor samples in the study. (68.5)
DAB [?]	54	29	8.4 GB	db (SQLite), duckdb, sql, bson (Mongo)	Average rating of all businesses located in Indianapolis, Indiana. (3.547)

Table 1: Benchmark suites. datagentbench files are database dumps served as SQLite/DuckDB files plus live PostgreSQL/MongoDB.

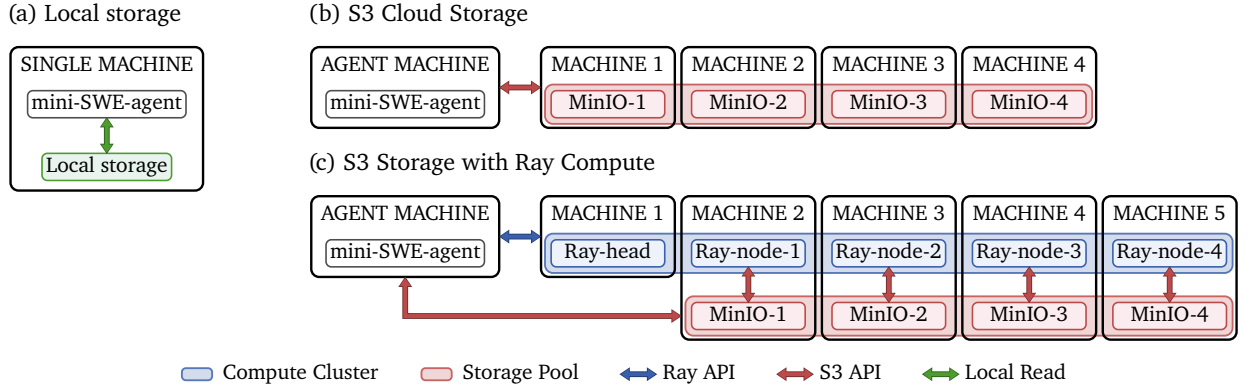


Figure 2: Diagram of System Architecture used for Experiments

TODO: update the writing accordingly

[Yongjoo: we selected benchmarks that are purely about data analysis, among these we are filtering for the criteria we want]

We evaluate on three benchmarks for data agents. LakeQA [22] is a multi-hop QA benchmark that requires agents to locate and combine evidence across a heterogeneous data lake. Due to cost constraints, we specifically use the LakeQA-mini suite, which spans 135 tasks over a 14.3GB data lake. KramaBench evaluates multi-step data-science QA across 104 tasks. Agents must locate relevant data across diverse formats and perform the necessary cleaning, transformation, and analysis to compute the answer. Lastly, DAB contains 54 tasks that test multi-database querying under realistic schema noise, requiring agents to locate relevant tables within large schemas and then resolve fuzzy joins and text extraction across separate database systems.

We select these benchmarks because each couples **data search** with task execution, reflecting real-world workflows where data must be located before analysis. Collectively, they span a diverse range of data formats, from plain text and tabular exports to scientific encodings and relational databases, reflecting the heterogeneity of real-world data. In addition, each benchmark supports deployment across all three of our storage environments (local disk, cloud storage, and cloud storage with co-located compute). Finally, all tasks can be completed through standard bash interfaces, which enables direct execution by agents such as mini-SWE-agent.

3.2. System Architecture (1 pg, excluding figure)

This section describes the system architecture used in our study. Our aim is to measure how the I/O costs of agentic data analysis are shaped by the underlying storage and execution infrastructure. To do this, we

evaluated three configurations shown in Figure 2 that reflect common industry deployments: local storage, cloud storage, and cloud storage with co-located compute. These configurations differ in where the data resides and where computation over the data can occur, which determines how many bytes must cross the boundary between storage and the agent. To isolate the effect of the storage and execution infrastructure itself, we use the same agent, mini-SWE-agent, across all three configurations. The remainder of this section describes each configuration in turn.

3.2.1. Local Storage

The local architecture serves as our baseline. Here, the data and agent are co-located on a single machine, and the data are stored as ordinary disk files. This allows the agent to read the data directly via the filesystem. Because the agent and data reside together, no bytes cross the network.

TODO: Describe OS cache. When we clear the cache.

TODO: STATE HOW IT IS ACTUALLY DEPLOYED (docker container)

3.2.2. Cloud Storage

Modern cloud analytics relies heavily on cloud storage systems such as Amazon S3, which physically separate compute engines from storage infrastructure [?]. We mirror this architecture using MinIO, an S3-compatible object storage server deployed across four storage nodes external to the agent environment. Without filesystem access, the agent must list and retrieve objects through the S3 API before any analysis can begin. This allows us to measure the network latency and data transfer costs incurred when compute and storage are disaggregated.

TODO: minio version, how much memory and CPU cores each component had access to, one-line description about minio's data placement,

TODO: examples of how the agent access data. `aws cp`, `aws ls`, Python streaming,

TODO: STATE HOW IT IS ACTUALLY DEPLOYED (docker container)

3.2.3. Cloud Storage with Co-Located Compute

Co-locating compute with cloud storage allows computation to move to data rather than data to computation. We co-locate the four-node MinIO deployment with a Ray cluster, running a compute worker on each storage node. This gives the agent two ways to reach the data: retrieve objects directly from S3 as in Section 3.2.2, or ship code to the cluster and compute in place, returning only the final result. Neither execution path is privileged, allowing us to evaluate whether the agent effectively leverages cluster computing. We measure changes in I/O costs between direct remote storage access (with local computation) and cluster-side execution.

4. Analysis & Findings

Our three architectures (A1 local, A2 object storage, A3 distributed cluster) hold the agent scaffold, the model, the prompt, and the task set fixed, and vary only where the bytes live and how the agent may reach them. This design lets us separate a question about *capability* from a question about *cost*: does moving the data change what the agent answers, or only what the answer costs to produce? We organize the analysis around one premise and four research axes, summarized in Table ?? . The premise (§4.3) establishes that accuracy is substrate-invariant, which is what licenses the remaining axes to treat cost, not correctness, as the dependent variable.

(Label check: §4 refers to the three arms as Local / S3 / Ray-Data. Once §3.2.3 is written, align these with the A1–A3 labels used in the methodology.)

4.1. Benchmark Accuracy is Largely Stable Across Storage Architectures.

We first examined whether task-solving accuracy remains consistent across varying system architectures, which are described in detail in §3.2. We assessed accuracy on the three benchmarks described in §3.1, LakeQA, KramaBench, and DAB, both overall as well as with per-task granularity.

Overall Benchmark Accuracy Storage architecture shapes how an agent accesses data to solve a task. Local storage allows the agent to read files directly in place, whereas with cloud storage it must first locate and fetch

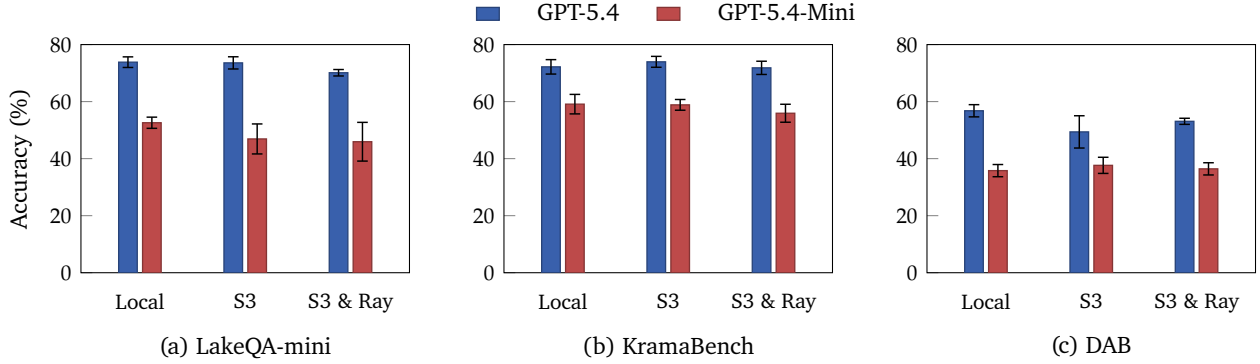


Figure 3: Accuracy by storage backend across three benchmarks. Error bars show $\pm$ one standard deviation across runs.

objects before analyzing them. As a result, the same task requires different data access trajectories depending on the storage architecture, which can in turn affect task accuracy. We therefore measure accuracy before evaluating I/O costs, as reduced I/O costs offer little benefit if task accuracy suffers. In addition, stable accuracy across architectures is necessary for a meaningful comparison of I/O costs. Gaps in accuracy could cause measured I/O costs to reflect variations in the work completed successfully rather than differences in the cost of solving the same task.

We evaluate mini-SWE-agent with GPT-5.4 and GPT-5.4-Mini across three benchmarks: LakeQA, KramaBench, and DAB. For each model–benchmark pair, we evaluate three system architectures: local storage, S3-compatible storage via MinIO, and a distributed cluster with MinIO object storage and Ray distributed compute. Each configuration is run three times and we report the mean task accuracy and standard deviation across runs in [Figure 3](#). All subsequent results presented in this paper are derived from these same experimental runs.

Accuracy is relatively consistent across storage architectures. On LakeQA-mini, GPT-5.4 scores 73.8% on local, 73.7% on S3, and 70.1% on S3 & Ray, while GPT-5.4-Mini scores lower at 52.6% on local, 46.9% on S3, and 45.9% on S3 & Ray ([Figure 3a](#)). For KramaBench, GPT-5.4 scored 72.2% on local, 74.0% on S3, and 71.8% on S3 & Ray, while GPT-5.4-Mini scored 59.1% on local, 58.9% on S3, and 55.9% on S3 & Ray ([Figure 3b](#)). For DAB, GPT-5.4 scored 56.8% on local, 49.4% on S3, and 53.1% on S3 & Ray, and GPT-5.4-Mini scored 35.8% on local, 37.7% on S3, and 36.4% on S3 & Ray ([Figure 3c](#)). In most cases, the differences between architectures fall within run-to-run variance. S3 & Ray trends slightly lower on average, scoring the lowest in four out of the six model–benchmark pairings. This gap exceeds run-to-run variance in only two cases, both with GPT-5.4. On LakeQA with GPT-5.4, S3 & Ray scores 3.5% below both local and S3, while on DAB it scores 3.7% below local. Across models, GPT-5.4 outperforms GPT-5.4-Mini in every condition.

We consider local storage to be the baseline, as it represents the most conventional mode of data access. Comparatively, S3 introduces additional steps to data access, so we anticipated a decline in task accuracy. However, accuracy remains largely consistent between local storage and S3. This suggests that the agent is capable of adapting its data access strategy to different storage architectures. Because S3 & Ray retains the same S3-based data access as S3 while additionally allowing the agent to submit jobs to Ray for remote analysis, its lower accuracy is unlikely to stem from restricted data access. Instead, it suggests that the agent does not make effective use of Ray, a behavior we examine in [Section 4.3](#). Overall, the consistency of accuracy across architectures provides a basis for comparing I/O costs.

Per Task Accuracy Variance [Figure 3](#) reports overall accuracy across storage architectures, but aggregate results do not show how performance varies for individual tasks. Agent behavior is nondeterministic, so the same task can succeed or fail across different runs under identical conditions. We therefore analyze accuracy at the task level. This allows us to examine whether task outcomes depend on storage architecture or vary across runs,

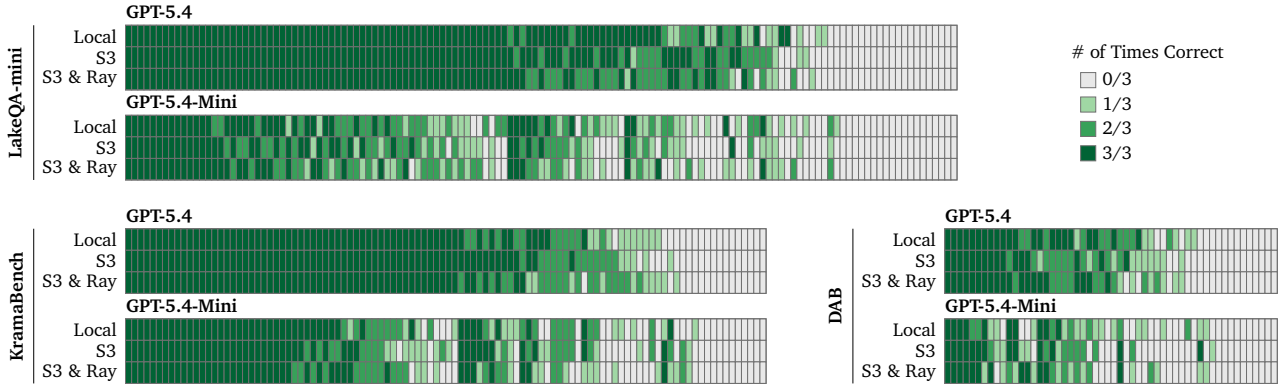


Figure 4: Per-task accuracy across storage architectures. Each cell shows the number of correct runs out of three. Columns are sorted by GPT-5.4’s total correct runs, and both models share the same column order.

as well as whether these patterns are consistent between benchmarks. In addition, task-level results allow for a more detailed comparison between GPT-5.4 and GPT-5.4-Mini, showing whether the models differ in which tasks they can solve, how reliably they solve them, or both.

As we did in Figure 3, we evaluated mini-SWE-agent with GPT-5.4 and GPT-5.4-Mini on LakeQA-mini, KramaBench, and DAB using our three storage architectures: Local, S3, and S3 & Ray. The results for each configuration are shown per task in Figure 4, where each cell shows the number of correct runs for a task. Gray indicates that a task was never solved, while increasingly darker green indicates that it was solved in one, two, or all three runs. Columns are sorted in descending order by the number of times GPT-5.4 solved the task for all three architectures, out of nine. Ties are broken by the corresponding count for GPT-5.4-Mini and then by task order. The heat maps for both models use the same column ordering, allowing the results to be compared task by task.

For each benchmark-model pair, tasks can be grouped by how consistently they are solved across the three storage architectures. For GPT-5.4, 62/135 tasks pass all nine runs for LakeQA-mini, 54/104 for KramaBench, and 10/54 for DAB, while GPT-5.4-Mini consistently passes 17/135 for LakeQA-mini, 30/104 for KramaBench, and 5/54 for DAB. GPT-5.4-Mini has more variability between runs, inconsistently passing (1-8 passes/9 runs) 92/135 LakeQA tasks, 58/104 KramaBench tasks, and 33/54 DAB tasks compared to 52/135 for LakeQA, 36/104 for KramaBench, and 31/54 for DAB for GPT-5.4. Of the 126 tasks passed consistently with GPT-5.4, 81 become inconsistent passes with GPT-5.4-Mini. Even for tasks passed inconsistently by both models, GPT-5.4-Mini succeeds less often, averaging 3.8/9 runs correct compared to 6.0/9 for GPT-5.4. Within the tasks that pass inconsistently, there are few tasks that are solved in every run for one architecture and no runs for another. There are 8 such tasks for GPT-5.4-Mini, with succeeding and failing architectures varying between tasks. For GPT-5.4, there are 7 such tasks, all failing for S3 & Ray.

Figure 4 shows that the lower accuracy observed for GPT-5.4-Mini in Figure 3 is driven more by model reliability than task coverage. Compared to GPT-5.4, GPT-5.4-Mini solves fewer tasks consistently and has a lower average accuracy on tasks passed inconsistently by both models. Despite these differences, the two models largely fail the same tasks. This indicates that GPT-5.4-Mini can solve many of the same tasks as GPT-5.4, but is less likely to reproduce these results across runs. These results are expected given that GPT-5.4-Mini is a more lightweight model designed for speed and cost efficiency. For GPT-5.4-Mini, tasks that pass inconsistently don’t show any patterns that favor one architecture over another, indicating that any differences in overall accuracy between architectures is likely due to run-to-run variance. GPT-5.4, on the other hand, has a few tasks that fail consistently on S3 & Ray while passing consistently on either Local, S3, or both. This matches the lower accuracy we saw for S3 & Ray in Figure 3, and is discussed further in Section 4.3.

4.2. Runtime Variance (use a full summary sentence)

Breakdown of Command Runtime In Figure 5 we look more closely at how the agent spends its runtime. Since S3 storage adds an extra data-loading step that is not present with local storage, this may increase command

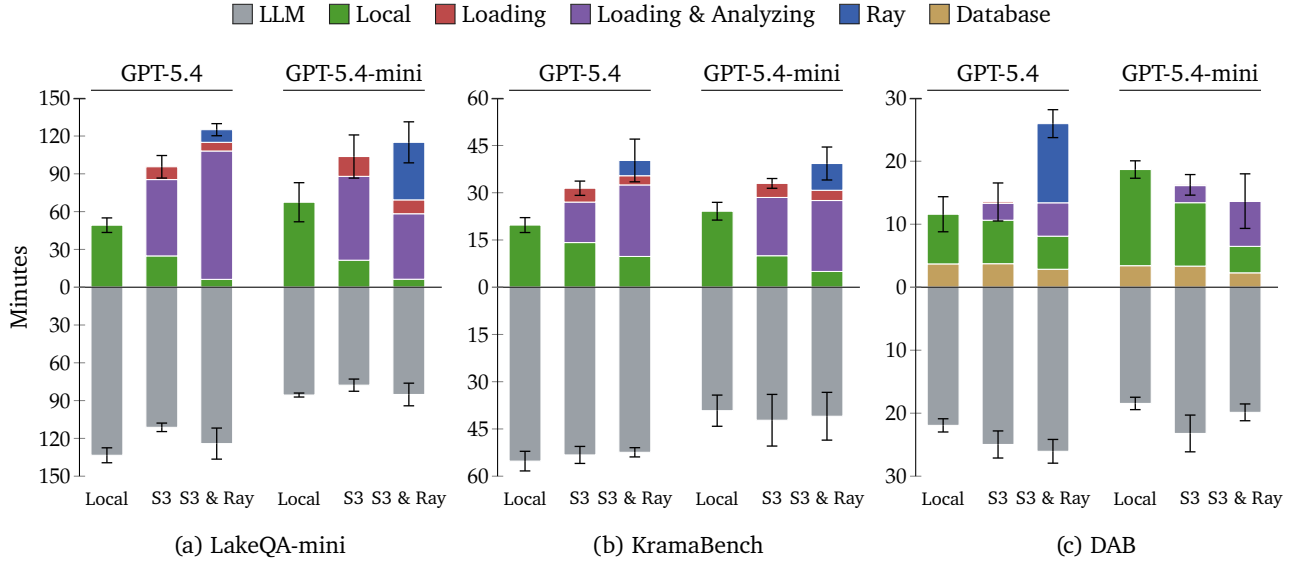


Figure 5: Command time by category and LLM latency for each model and architecture. Bars show the mean across three runs and error bars show $\pm$ one standard deviation.

runtime. The addition of Ray could offset these costs by running computation near the data and in parallel, but it could also introduce scheduling and coordination overhead. As a result, we examine how these changes in storage architecture affect command runtime and distribution. The latency for LLM responses also accounts for a substantial share of runtime, which may limit how much faster commands can improve overall speed. We separate LLM latency from command execution time to see how much each contributes.

Figure 5 presents the breakdown of command runtime. We measured the wall-clock time of each agent command and classified it by the type of work performed. Local compute includes commands that operate on data stored on the local disk (e.g., `grep -c 'median income' county.txt`). Loading includes commands that only fetch or list data (e.g., `aws s3 cp s3://bucket/rows.txt rows.txt`), while Loading & Analyzing includes commands that do both fetching and processing (e.g., `aws s3 cp s3://bucket/meta.txt - | grep 'school year'`). For the S3 & Ray storage, Ray jobs captures work submitted to the Ray cluster (e.g., `ray job submit - python job.py`). The database category includes queries to MongoDB or PostgreSQL live databases (e.g., `psql -c "SELECT count(*) FROM reviews"`) which are used for the DAB benchmark. For each configuration, we averaged the command time in each category across the three runs, then plotted them as a stacked bar. Error bars indicate one standard deviation of the total command across the three runs. Below the x axis, we show the average LLM latency, along with its standard deviation.

Under S3 and S3 & Ray, Loading and Loading & Analyzing together accounted for a substantial share of command runtime (55-87% for LakeQA-mini, 55-70% for KramaBench, and 17-53% for DAB). Between the two, Loading & Analyzing accounted for most of this time in all S3-based configurations, with negligible Loading runtime for DAB. Local compute time fell from Local to S3 to S3 & Ray across all six model-benchmark pairs, with the largest decrease for GPT-5.4-Mini on LakeQA-mini (67.5 to 21.4 to 6.3 minutes). However, total command runtime increased across the three architectures for five of the six model-benchmark pairs. From Local to S3, the largest increase was 46.4 minutes for GPT-5.4 on LakeQA-mini, while the smallest was 1.9 minutes for GPT-5.4 on DAB. From S3 to S3 & Ray, the largest increase was for GPT-5.4 on LakeQA-mini, at 29.4 minutes, while the smallest was 6.3 minutes for GPT-5.4-Mini on KramaBench. The exception to this trend was GPT-5.4-Mini on DAB, where command time fell from 18.7 to 16.2 to 13.7 minutes, with overlapping intervals between adjacent architectures. Overall, Ray usage was highly variable. At the maximum, Ray accounted for 48.6% of command runtime for GPT-5.4 on DAB, while GPT-5.4-Mini did not use Ray at all on DAB. LLM latency accounted for 43-74% of the total runtime, exceeding command runtime in 14/18 configurations. Latency for GPT-5.4-Mini was consistently lower compared to GPT-5.4, with the largest gap for LakeQA-mini under Local (85.6 versus 133.4 minutes).

Storage	Search		Load		Examine		Compute	
Local	Total	41.2% $\pm$ 1.5	Total	0.0% $\pm$ 0.0	Total	40.7% $\pm$ 1.2	Total	25.7% $\pm$ 1.1
	ls+sed	16.3% $\pm$ 1.1	none	—	python	11.2% $\pm$ 0.2	python	23.0% $\pm$ 1.4
	ls	11.9% $\pm$ 0.1	(read in place)		sed	9.4% $\pm$ 0.5	python+sed	1.2% $\pm$ 0.3
	ls+grep+sed	5.5% $\pm$ 0.6			ls+sed	7.7% $\pm$ 0.8	ls+python	0.4% $\pm$ 0.1
S3	Total	31.9% $\pm$ 1.0	Total	34.9% $\pm$ 1.0	Total	46.9% $\pm$ 0.8	Total	19.4% $\pm$ 1.3
	aws ls+grep	14.4% $\pm$ 0.8	aws cp+python	10.3% $\pm$ 0.3	python	6.3% $\pm$ 0.7	python	14.4% $\pm$ 0.8
	aws ls	5.2% $\pm$ 0.1	aws cp+sed	5.9% $\pm$ 0.4	aws cp+python	6.2% $\pm$ 0.2	aws cp+python	1.7% $\pm$ 0.3
	aws ls+grep+sed	4.3% $\pm$ 0.2	aws cp+grep+sed	4.4% $\pm$ 0.4	aws cp+sed	5.9% $\pm$ 0.4	python+sed	1.4% $\pm$ 0.2
Ray Data	Total	29.4% $\pm$ 1.0	Total	54.9% $\pm$ 2.2	Total	54.0% $\pm$ 1.5	Total	24.6% $\pm$ 0.8
	aws ls+grep+sed	10.6% $\pm$ 0.3	python	14.5% $\pm$ 1.6	aws cp+sed	12.2% $\pm$ 0.7	python	15.8% $\pm$ 1.2
	aws ls+grep	8.2% $\pm$ 0.7	aws cp+sed	12.2% $\pm$ 0.7	aws ls+grep	5.4% $\pm$ 0.6	ray submit	1.8% $\pm$ 0.2
	aws ls	2.0% $\pm$ 0.1	aws cp+python	7.3% $\pm$ 0.4	aws cp+python	4.9% $\pm$ 0.4	python+sed	1.5% $\pm$ 0.3

Table 2: Command patterns by category for each storage architecture (LakeQA-mini, GPT-5.4). Values are percentages of all commands, averaged over three runs; totals include the standard deviation and can sum to more than 100% because commands can fall into multiple categories.

The increase in total command runtime for S3 and S3 & Ray is consistent with the added step of fetching data from S3 storage. This effect was most pronounced on file-based benchmarks LakeQA-mini and KramaBench, where all the data were moved to S3. By contrast, only some of the data for DAB had to be fetched from S3 (PostgreSQL and MongoDB were always hosted locally), so its command runtime did not change considerably from Local to S3. Runtime spent loading data from S3 predominantly fell into Loading & Analyzing, indicating that the agent frequently retrieved and analyzed data in one command. This tendency explains the decrease in Local compute for S3 and S3 & Ray. For Local, searching a file (`grep -n -i 'capital' /data/wikipedia/California/content.txt`) falls under Local compute. With S3, the same search is combined with a file download, (`aws s3 cp s3://datalake-qa/wikipedia/California/content.txt ./California.txt && grep -n -i 'capital' ./California.txt`) and is reclassified as Loading & Analyzing. Looking at S3 compared to S3 & Ray, we expected Ray to shorten command time by shifting analysis to the cluster, reducing local computation and data transfer to the agent. Local compute did decrease, but in five of the six S3 & Ray configurations this was paired with increases in the Loading & Analyzing category, indicating that compute was not actually pushed to Ray. In addition, for all five configurations that used Ray (GPT-5.4-Mini had 0 Ray use on DAB), Ray command time exceeded any time saved in other categories, causing overall command runtime to increase. As expected, LLM latency was a significant portion of the runtime for both models. Latency was lower for GPT-5.4-Mini across all configurations, consistent with its design as a faster, smaller model.

Breakdown of Commands Used In Table 2 we build on the command runtime breakdown from the previous section by examining the specific commands the agent uses per storage architecture. We identify which commands the agent adapts when the data moves from local into S3, as well as which commands change when Ray compute is added. Changes in commands highlight the parts of task evaluation that are most dependent on the storage architecture as well as parts that are storage-agnostic. For S3 & Ray, we also evaluate whether the agent changes its commands to take advantage of the Ray cluster. To keep our analysis concise, we look only at commands from LakeQA-mini runs with GPT-5.4. We chose LakeQA-mini because it is the most search heavy of our benchmarks. Because searching depends on listing and previewing files, which function differently for local storage compared to S3, these tasks give the agent the most opportunity to vary its command choices.

To compare how the agent performs different types of work across storage architectures, we sort commands into four categories. Search commands use file names and metadata to locate relevant data, while Load commands pull objects from S3 storage. Examine commands read the contents of the data as they are, for example to preview a file or find a specific line. Compute commands process the data to calculate new values, such as through aggregations or table joins. Criteria for assigning commands into these categories were developed by looking at recurring patterns in how the commands were used in different contexts. For example, when `grep` is used to search a list of file names, it’s a Search command, but when it searches the inside of a file, it gets

categorized as Examine. Because the agent often chains several steps into a single command, one command can be in more than one category. For example, a command that copies a CSV file from S3 with `aws s3 cp` and then runs `pd.read_csv(f).groupby("region").sum()` on the copied file counts as both Load and Compute, while streaming an object with `aws s3 cp ... - piped into sed` to preview it counts as both Load and Examine. As a result, the category shares for a storage architecture in Table 2 can total to more than 100%. A pattern records which tools a command combines, not what they do, so the same pattern can appear in more than one category. For example, `aws cp+python` is Load and Examine when the Python code prints a few rows of the copied file, but Load and Compute when it aggregates the data. For each category, we report the top three command patterns and their percentage share of all commands for that architecture. We compute the percentages separately for each run and report the mean and standard deviation across runs.

Search accounts for a smaller share of commands for S3 and S3 & Ray (31.9% and 29.4%) compared to Local storage (41.2%). For Local storage, the most common Search patterns all include `ls`; this is replaced with `aws s3 ls` for both S3 architectures. Examine follows the opposite trend, increasing from 40.7% under Local storage to 46.9% under S3 and 54.0% under S3 & Ray. For Examine, Python and `sed` are commonly used across architectures, but for S3 and S3 & Ray they often appear alongside `aws s3 cp`, combining data retrieval and examination in one command. Load increases from 34.9% of commands under S3 to 54.9% under S3 & Ray, alongside a change in its leading command patterns. For S3, all three most common Load patterns include `aws s3 cp`, while Python alone ranks first under S3 & Ray, accounting for 14.5% of all commands. The sum of the category percentages rises from 107.6% for Local to 133.1% under S3 and 162.9% under S3 & Ray, indicating an increase in commands that span multiple categories. For example, `aws cp+sed` falls in both Load and Examine, accounting for 5.9% of S3 commands and 12.2% S3 & Ray commands. Compute varies less across architectures than other categories, accounting for a similar share in Local Storage (25.7%) and S3 & Ray (24.6%), while dropping slightly in S3 (19.4%). Python is the most common Compute command across all three architectures, while `ray submit` was second for S3 & Ray at 1.8% of commands.

The changes in Search commands reflect how different commands are required for different storage architectures. Under Local storage, the agent lists one directory level per `ls` call and needs several calls to reach a nested dataset. Under S3 and S3 & Ray, it instead uses `aws s3 ls -recursive` to list the names of every object at once, often filtering the output with `grep`. This allows the agent to find the data faster with S3 and S3 & Ray, decreasing the portion of commands that are needed for Search. On the other hand, there are more Examine commands for S3 and S3 & Ray because the agent must retrieve the data from S3 before it can look at it. It often retrieves and previews data in one command, such as `textttaws s3 cp piped into sed`. Since many Load commands also performs another step, more commands fall into multiple categories under S3 and S3 & Ray. The further increase in overlap under S3 & Ray reflects the agent’s preferred data access methods. Load patterns show a shift towards standalone Python under S3 & Ray, with loading via Python accounting for 14.5% of all commands. On the other hand, all three leading patterns under S3 include `aws s3 cp`. These Python commands retrieve data from S3 as well as Examine or execute Compute on it, thereby falling into multiple categories. Rather than showing the influence of Ray compute, this behavior may be due to differences in prompting. The S3 & Ray prompt demonstrates `pd.read_csv("s3://...")` in its Ray job examples, which may encourage this reading method in local Python commands as well. Greater overlap is also visible in `aws cp+sed`, which performs both Load and Examine. Its share rises from 5.9% of all commands under S3 to 12.2% under S3 & Ray, showing that combined retrieval and examination becomes more common. These changes all involve how the agent searches and loads data, but the calculations required to answer the questions remains similar. Accordingly, Compute varies the least, and Python remains its most common pattern across all three architectures. Even with the option to use the cluster, Ray job submissions account for only 1.8% of commands under S3 & Ray.

Agent Trajectory Differs by Model While the previous figures focus on the effects of changing storage architectures, in Figure 6 we examine how the choice of model influences the agent trajectory. We compare GPT-5.4 and GPT-5.4-Mini trajectories to evaluate whether the two models organize their work differently across steps. In addition, we compare trajectories from LakeQA-mini, KramaBench, and DAB to see how the style of task influences how the agent’s approach of solving it. Because LakeQA-mini, KramaBench, and DAB all vary in their data types as well as the balance of searching vs processing, their respective tasks may call for different execution patterns.

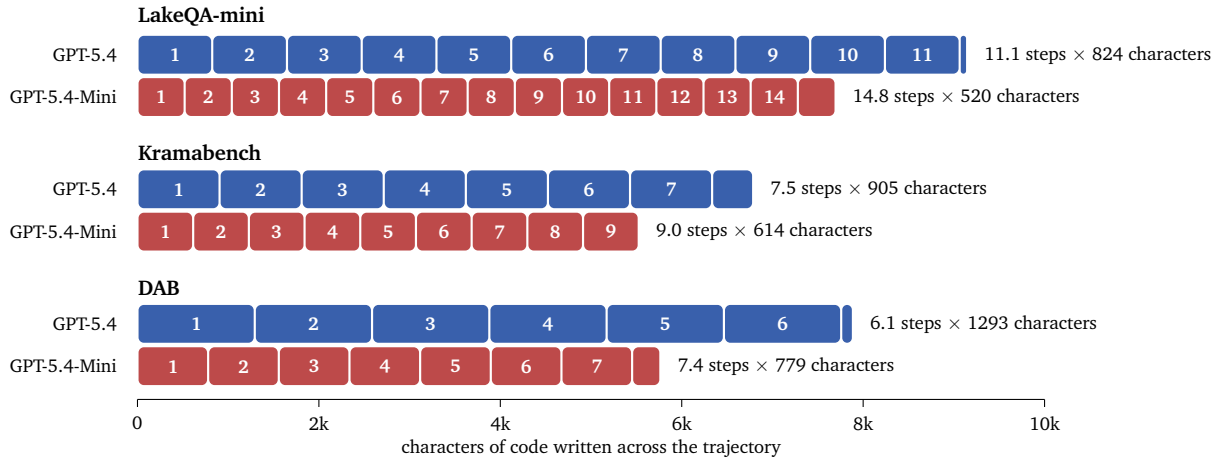


Figure 6: Average trajectory length and command character counts across models and benchmarks. Each block represents a step on the trajectory, with block width proportional to the mean character count per command.

To represent an agent trajectory, we look at two characteristics: the number of steps the agent takes, and the character count of the commands issued for each step. For each model-benchmark pair, we calculate the average step count across all the trajectories and the average character count across all the commands in those trajectories. Both measures showed similar trends across the three storage architectures, so we take the average across all nine runs for a model-benchmark pair instead of separately per architecture. Results are shown in Figure 6, where each box represents a step on the agent trajectory. Box width is drawn to scale based on the average command character count.

On all three benchmarks, GPT-5.4-Mini took more steps per trajectory compared to GPT-5.4, but wrote less code per step. For LakeQA-mini, GPT-5.4 took 11.1 steps/trajectory with 824 characters/step, while GPT-5.4-Mini took 14.8 steps/trajectory with 520 characters/step. On KramaBench, GPT-5.4 took 7.5 steps/trajectory with 905 characters/step and GPT-5.4-Mini took 9.0 steps/trajectory with 641 characters/step. For DAB, GPT-5.4 averaged 6.1 steps/trajectory with 1,293 characters/step, while GPT-5.4-Mini averaged 7.4 steps/trajectory and 779 characters/step. Despite taking more steps/trajectory on average, GPT-5.4-Mini writes less code in total on all three benchmarks. Across benchmarks, both models showed consistent trajectory patterns: LakeQA-mini took more, shorter steps, DAB uses fewer, longer steps, and KramaBench lies between the two for both step count and command length.

The combination of longer commands and fewer steps for GPT-5.4 makes sense, since doing more work per step decreases the total number of steps needed. In particular, the complete command length distribution (Figure 9) reveals that compared to GPT-5.4-Mini, GPT-5.4 writes significantly more commands exceeding 1,000 characters. These commands often contain a complete program, from loading data to printing an answer, allowing the agent to execute several operations in one step. On the other hand, GPT-5.4-Mini appears to build a solution by incrementally writing shorter commands, resulting in more steps/trajectory. One possible explanation for this is that writing and checking smaller pieces is easier for GPT-5.4-Mini than producing a long, correct program in one attempt. Still, GPT-5.4-Mini writes less code in total, so its extra steps do not fully make up for the shorter commands. The difference in trajectories between benchmarks can be explained by the different task styles. LakeQA-mini is the most search heavy, requiring the agent to use many short search and inspection commands to solve the task. Comparatively, DAB requires less search is mostly SQL-based, so a long query can perform substantial computation in one step. KramaBench falls between them: its known input directory requires less searching, but Python cleaning and transformation still take multiple steps.

4.3. Data Placement and Movement

Data loading methods vary by file size In Figure 7 we examine the methods used by the agent to load data from S3 storage. Depending on how it plans to use the data, the agent can access files in several different ways.

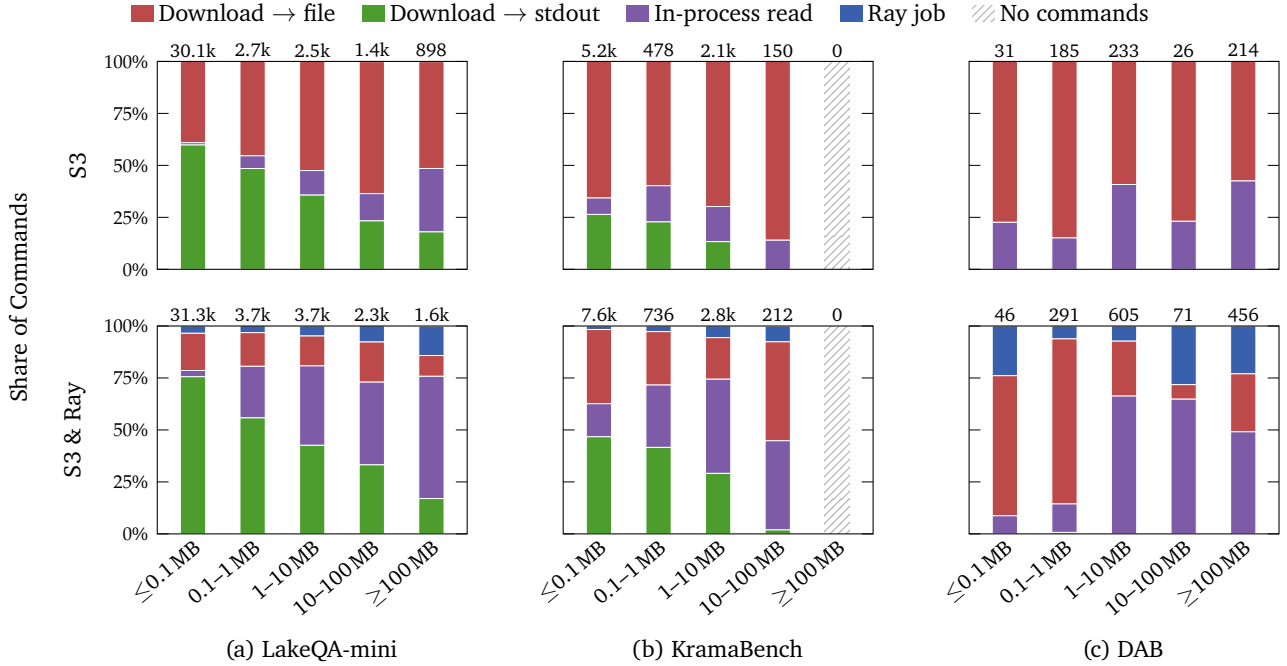


Figure 7: Share of commands by read method within each file-size bin, per data setup (rows) and benchmark (subfigures). Numbers above the bars give the number of commands in each bin; hatched bins contain no commands.

Downloading a file to local disk allows it to use bash tools and reuse the file in later commands. Streaming to standard output supports quick inspection, such as viewing the first few rows. For Python analysis, the agent can read the data directly into memory, and when Ray compute is added, it can also submit a job to workers paired with the storage nodes and receive only the result. These methods support different kinds of work and move data in different ways, potentially affecting I/O costs. We examine how the loading methods vary with file size, as well as how the behavior changes across benchmarks. Each benchmark has different file types and task requirements, so they may call for different methods of data loading.

We analyze the commands used by both GPT-5.4 and GPT-5.4-Mini to load data in S3 and S3 & Ray runs, excluding local runs where no data loading step is needed. Each object read was classified into one of four loading methods according to where and how the data were loaded. Download → file saves the object to a local path, as in `aws s3 cp s3://.../rows.txt inspect/rows.txt`. Download → stdout streams the object into another command without saving a file, for example `aws s3 cp s3://.../metadata.txt - | sed -n '1,220p'`. In-process reads open an `s3://` URI within a Python library, such as DuckDB query `FROM read_csv('s3://.../rows.txt')` or pandas `pd.read_csv('s3://.../data.csv')`. A Ray job reads the object within a script submitted through `ray job submit`. We classify individual reads instead of commands because a single command can combine multiple loading methods, such as `aws s3 cp s3://.../content.txt buffalo.txt` followed by `pd.read_csv('s3://.../rows.txt')`. By parsing the file paths from object reads, we identify the accessed files and group the reads by object size. Results are presented in Figure 7, which shows the percentage of object reads that used each loading method per size bin. KramaBench has no files larger than 100MB, so we leave the column blank.

Most reads for LakeQA-mini and KramaBench are on small files, with files less than 0.1 MB accounting for more than 65% of object reads. On DAB, most reads access files 0.1-10 MB, with files of at least 100 MB accounting for about a third of the reads. Across all benchmarks and storage architectures, the share of in-process reads trended upwards with file size. The largest increase can be seen with LakeQA-mini and the S3 & Ray architecture, where in-process reads are just 3.0% of for files less than 0.1 MB compared to 58.8% for files larger than 100

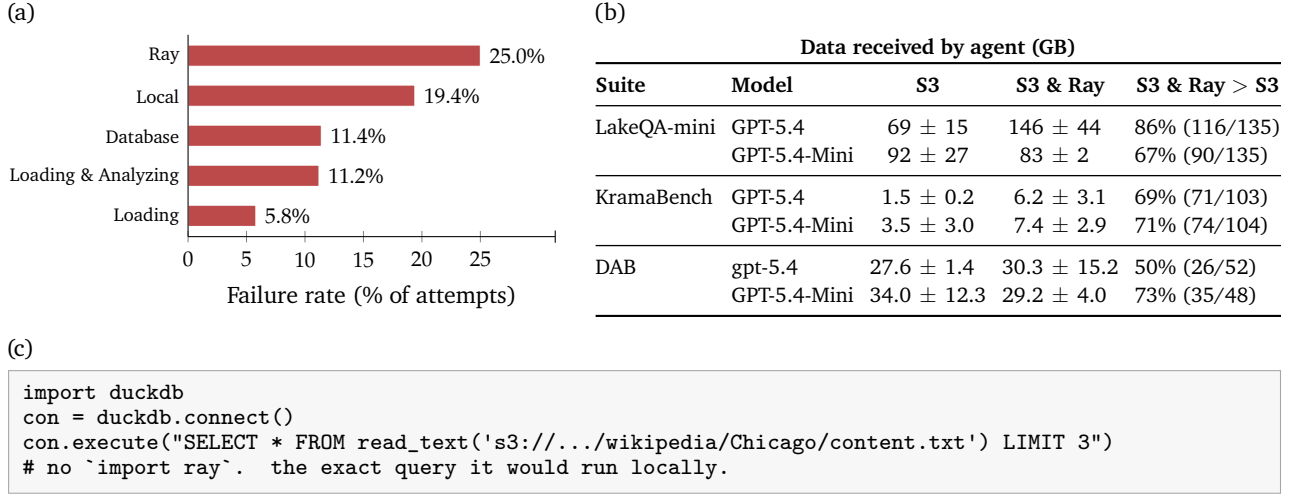


Figure 8: Data-access reliability. **a** Failure rate of each data-access approach, as a share of that approach’s attempts. **b** Data received by the agent (GB) with S3 versus Ray Data for each suite and model, as mean $\pm$ standard deviation; Ray > S3 is the share of tasks where the Ray Data value is higher than the S3 value. **c** An example command shipped as a Ray job: a plain DuckDB query that never uses Ray.

MB. Streaming to stdout showed the opposite trend. For LakeQA-mini and KramaBench, its share decreases steadily as file size grows, with the most sharp drop on LakeQA-mini on Ray Data from 75.6% for files ≤ 0.1 MB to 17.0% for files ≥ 100 MB. On S3, download methods remain the most common in every bin while on Ray-Data, in-process reads become the most common method to read larger files on all three benchmarks. When comparing S3 to S3 & Ray, the share of commands that downloads to a file falls in every bin across the graphs, with the largest drop from 76.9% to 7.0% for files 10-100 MB.

Small files are read mostly through download methods, either streamed to stdout or downloaded to a file depending on the benchmark. On S3, download methods remain the most common in every bin. On Ray Data, in-process reads become the most common method for larger files in all three benchmarks: from 10 MB in LakeQA-mini, from 1 MB in DAB, and at 1–10 MB in KramaBench. Moving from S3 to Ray Data, the share of downloads to file falls in every bin of every benchmark. The largest drop is in DAB at 10–100 MB, from 76.9

Ray Data Use

Why this matters. A null result on accuracy is the enabling condition for the rest of the paper. It means the substrate is not a confound on task difficulty, so any difference we observe downstream is a difference in *how* the same answer was reached. It also sharpens the argument of Kapoor et al. [10]: two systems that are indistinguishable on the leaderboard here differ substantially on every cost dimension we measure.

5. Conclusion

[Haocheng: load occupied a large proportion of the total time cost, whether can we analyze the possible benefits from prefetching? it is possible to hide the latency for RayData backend on some benchmarks]

References

- [1] Rajeev Balasubramonian, Jichuan Chang, Troy Manning, Jaime H. Moreno, Richard Murphy, Ravi Nair, and Steven Swanson. Near-data processing: Insights from a micro-46 workshop. *IEEE Micro*, 34(4):36–42, 2014. doi: 10.1109/MM.2014.64.
- [2] Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *International Conference on Learning Representations (ICLR)*, 2025.
- [3] CrewAI. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. <https://github.com/crewAIInc/crewAI>, 2024. Accessed: 2026-08-13.
- [4] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI)*, pages 137–150, San Francisco, CA, 2004. USENIX Association.
- [5] Jaeyoung Do, Yang-Suk Kee, Jignesh M. Patel, Chanik Park, Kwanghyun Park, and David J. DeWitt. Query processing on smart ssds: Opportunities and challenges. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pages 1221–1230, 2013. doi: 10.1145/2463676.2465295.
- [6] Alex Egg, Martin Iglesias Goyanes, Friso Kingma, Andreu Mora, Leandro von Werra, and Thomas Wolf. DABstep: Data agent benchmark for multi-step reasoning. *arXiv preprint arXiv:2506.23719*, 2025.
- [7] Xueyu Hu, Ziyu Zhao, Shuang Wei, Ziwei Chai, Qianli Ma, Guoyin Wang, Xuwu Wang, Jing Su, Jingjing Xu, Ming Zhu, Yao Cheng, Jianbo Yuan, Jiwei Li, Kun Kuang, Yang Yang, Hongxia Yang, and Fei Wu. InfiAgent-DABench: Evaluating agents on data analysis tasks. In *International Conference on Machine Learning (ICML)*, 2024.
- [8] Yiming Huang, Jianwen Luo, Yan Yu, Yitong Zhang, Fangyu Lei, Yifan Wei, Shizhu He, Lifu Huang, Xiao Liu, Jun Zhao, and Kang Liu. DA-Code: Agent data science code generation benchmark for large language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 13487–13521, 2024.
- [9] Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. DSbench: How far are data science agents from becoming data science experts? In *International Conference on Learning Representations (ICLR)*, 2025.
- [10] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=Zy4uFzMviZ>.
- [11] Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S. Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndzomga, Dheeraj Oruganty, Sophie Luskin, Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/forum?id=vUaY1t64ZZ>.
- [12] Eugenie Lai, Gerardo Vitagliano, Ziyu Zhang, Om Chabra, Sivaprasad Sudhir, Anna Zeng, Anton A. Zabreyko, Chenning Li, Ferdi Kossman, Jialin Ding, J. Chen, M. Markakis, M. Russo, W. Wang, Z. Wu, M. Cafarella, L. Cao, S. Madden, and T. Kraska. KramaBench: A benchmark for AI systems on data-to-insight pipelines over data lakes. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2506.06541; OpenReview fZfUdeCC5X.
- [13] Jiale Lao, Andreas Zimmerer, Olga Ovcharenko, Tianji Cong, Matthew Russo, Gerardo Vitagliano, Michael

- Cochez, Fatma Özcan, Gautam Gupta, Thibaud Hottelier, H. V. Jagadish, Kris Kissel, Sebastian Schelter, Andreas Kipf, and Immanuel Trummer. SemBench: A benchmark for semantic query processing engines. *arXiv preprint arXiv:2511.01716*, 2025.
- [14] Fangyu Lei, Jinxiang Meng, Yiming Huang, Junjie Zhao, Yitong Zhang, Jianwen Luo, Xin Zou, Ruiyi Yang, Wenbo Shi, Yan Gao, Shizhu He, Zuo Wang, Qian Liu, Yang Wang, Ke Wang, Jun Zhao, and Kang Liu. DComp: Benchmarking data agents across the full data intelligence lifecycle. In *International Conference on Learning Representations (ICLR)*, 2026. OpenReview EtzJy9yI5J.
- [15] Shu Lin, Arunprasad P. Marathe, Per-Åke Larson, Chong Chen, Calvin Sun, Paul Lee, and Weidong Yu. Near data processing in taurus database. *arXiv preprint arXiv:2506.20010*, 2025.
- [16] Yuyu Luo, Guoliang Li, Ju Fan, and Nan Tang. Data agents: Levels, state of the art, and open problems. In *Companion of the International Conference on Management of Data (SIGMOD Companion)*, pages 571–579. ACM, 2026.
- [17] Ruiying Ma, Shreya Shankar, Ruiqi Glen Chen, Yiming Lin, Sepanta Zeighami, Rajoshi Ghosh, Abhinav Gupta, Anushrut Gupta, Tanmai Gopal, and Aditya G. Parameswaran. Can AI agents answer your data questions? a benchmark for data agents. *arXiv preprint arXiv:2603.20576*, 2026.
- [18] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 561–577. USENIX Association, 2018.
- [19] Aymeric Roucher, Albert Villanova del Moral, Thomas Wolf, Leandro von Werra, and Erik Kaunismäki. smolagents: A barebones library for agents. <https://github.com/huggingface/smolagents>, 2025. Accessed: 2026-08-13.
- [20] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [21] Michael Solodko, Steven Gong, Guangwei Yu, Satya Krishna Gorti, Jesse C. Cresswell, and Victor Zhong. LakeQuest: Benchmarking ai data agents on natural language discovery and insight generation over data lakes. *arXiv preprint arXiv:2607.12310*, 2026.
- [22] Haonan Wang, Jiaxiang Liu, Yurong Liu, Austin Senna Wijaya, Tianle Zhou, Eden Wu, Yijia Chen, Wanting You, Reya Vir, Daniela Pinto, Grace Fan, Yusen Zhang, Juliana Freire, and Eugene Wu. LakeQA: Exploratory question answering over a million-scale data lake. In *International Conference on Machine Learning (ICML)*, 2026.
- [23] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning (ICML)*, 2024.
- [24] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.
- [25] Austin Senna Wijaya, Jiaxiang Liu, Haonan Wang, and Eugene Wu. SANA: What matters for QA agents over massive data lakes? *arXiv preprint arXiv:2606.13904*, 2026.
- [26] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [27] John Yang, Carlos E. Jimenez, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. mini-swe-agent. <https://github.com/SWE-agent/mini-swe-agent>, 2025. Accessed: 2026-08-13.

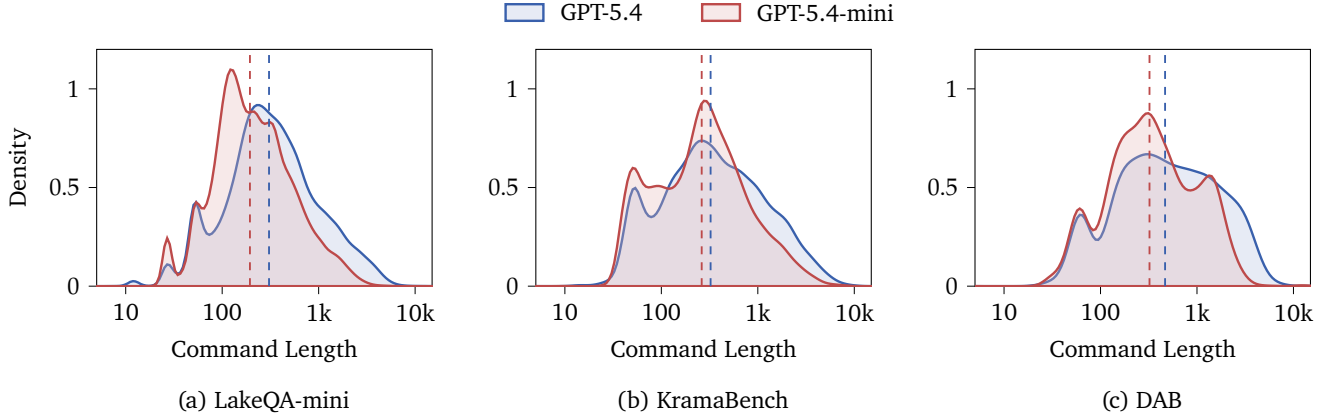


Figure 9: Distribution of command lengths (characters) for GPT-5.4 and GPT-5.4-mini on each benchmark (Gaussian kernel density estimate over $\log_{10}$ length). Dashed lines mark the median.

- [28] Yifei Yang, Xiangyao Yu, Marco Serafini, Ashraf Aboulnga, and Michael Stonebraker. Enhancing computation pushdown for cloud olap databases. *arXiv preprint arXiv:2312.15405*, 2023.
- [29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [30] Xiangyao Yu, Matt Youill, Matthew Woicik, Abdurrahman Ghanem, Marco Serafini, Ashraf Aboulnga, and Michael Stonebraker. Pushdowndb: Accelerating a dbms using s3 computation. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, 2020. doi: 10.1109/ICDE48307.2020.00174.
- [31] Matei Zaharia, Dhruba Borthakur, Joydeep Sen Sarma, Khaled Elmeleegy, Scott Shenker, and Ion Stoica. Delay scheduling: A simple technique for achieving locality and fairness in cluster scheduling. In *Proceedings of the 5th European Conference on Computer Systems (EuroSys)*, pages 265–278, 2010. doi: 10.1145/1755913.1755940.
- [32] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 15–28. USENIX Association, 2012.
- [33] Qiaohong Zhang, Weihao Ye, Jialong Chen, Yi Luo, BoYuan Li, Bowen Deng, Zibin Zheng, Jianhao Lin, Wei-Shi Zheng, and Chuan Chen. DataClawBench: An agent benchmark for exploratory real-world financial data analysis. *arXiv preprint arXiv:2605.02503*, 2026.
- [34] Robert Zhang, Eric Hayden Campbell, Dixin Tang, and Işıl Dillig. Optimal predicate pushdown synthesis. *Proceedings of the ACM on Programming Languages*, 10(PLDI):46, 2026. doi: 10.1145/3808312.
- [35] Yuxin Zhang, Ju Fan, Meihao Fan, Shaolei Zhang, and Xiaoyong Du. CoDA-Bench: Can code agents handle data-intensive tasks? In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2606.15300.